

Budget-Aware Tracerouting: Reducing Traceroute Probe Cost Through Adaptive Per-Hop Budget Allocation

Bakri Diyaolu

*Mathematics, Computer, and
Information Sciences
Mississippi Valley State University
Itta Bena, MS, USA
bakri.diyaolu@mvsu.edu*

Adin Lindsey

*Mathematics, Computer, and
Information Sciences
Mississippi Valley State University
Itta Bena, MS, USA
adin.lindsey@mvsu.edu*

Jeremiah Adderley

*Mathematics, Computer, and
Information Sciences
Mississippi Valley State University
Itta Bena, MS, USA
jeremiah.adderley@mvsu.edu*

Khaled Sabahein

*Mathematics, Computer, and
Information Sciences
Mississippi Valley State University
Itta Bena, MS, USA
khaled.sabahein@mvsu.edu*

Latonya Garner-Jackson

*Mathematics, Computer, and
Information Sciences
Mississippi Valley State University
Itta Bena, MS, USA
lgarner@mvsu.edu*

Abstract—Traditional traceroute tools send a fixed number of probes per hop, leading to redundant measurements and inefficient use of network resources. This research introduces a budget-aware traceroute system (BAT) designed to optimize probing efficiency while preserving accurate path discovery. Using Scamper as the probing engine, BAT dynamically allocates probe “credits” across hops based on per-hop confidence levels. This work highlights the potential for adaptive network measurement systems that balance precision, responsiveness, and bandwidth efficiency in large-scale Internet topology mapping.

Keywords—traceroute, network measurement, adaptive probing, budget-aware probing, probe reduction, Internet topology, Paris traceroute

I. INTRODUCTION

Network path tracing is fundamental to Internet diagnostics, topology mapping, and performance monitoring. Traditional traceroute sends a fixed number of probes at every hop regardless of response consistency, wasting resources on stable hops while potentially under-sampling unstable or load-balanced paths. As Internet-scale measurements grow increasingly common across research and industry, the cumulative cost of redundant probing becomes significant—contributing to unnecessary network overhead, increased measurement time, and excessive resource consumption. These inefficiencies are particularly problematic in large-scale measurement campaigns where thousands of destinations must be traced regularly, making it essential to develop smarter probing strategies that minimize waste without compromising the quality of collected path data.

Recent adaptive measurement techniques demonstrate that intelligent probe allocation can reduce overhead without sacrificing accuracy. However, most existing optimizations target individual components of the traceroute process rather than providing holistic resource management across the entire trace. This research implements a budget-constrained Paris Traceroute that treats probes as limited resources to be allocated strategically across hops. Our credit-based framework dynamically adapts to response patterns—stopping early at stable hops and redistributing remaining budget where needed. Evaluated against 100 diverse targets spanning DNS resolvers, CDN edges, cloud platforms, and

major web services, the system achieves comparable hop-resolution accuracy while significantly reducing total probe counts—critical for scalable measurement campaigns.

II. BACKGROUND & RELATED WORK

A. Traceroute

Traceroute is one of the most widely used tools for discovering Internet paths. Originally written by Van Jacobson in 1989 [1], it works by sending probe packets with incrementally increasing Time-to-Live (TTL) values. Each router along the path decrements the TTL field; when it reaches zero, the router discards the packet and returns an ICMP Time Exceeded message to the source, revealing the router's IP address. By increasing the TTL from 1 up to some maximum, traceroute progressively uncovers each hop along the forwarding path to a destination.

In its conventional form, traceroute sends a fixed number of probes per hop — typically three — regardless of how quickly or reliably each hop responds. For a maximum TTL of $h_{\max}$ and q probes per hop, the total probe cost is:

$$C = q \times h_{\max}$$

For example, with $q=3$ and $h_{\max}=32$, a single trace costs 96 probes, even if the destination is only 10 hops away and most hops respond consistently on the first attempt. At the scale of Internet-wide measurement campaigns, such as those conducted by CAIDA's Archipelago (Ark) infrastructure [2], this fixed-cost model generates enormous volumes of redundant probes, consuming time and network resources.

B. Paris Traceroute and ECMP

Equal-Cost Multipath (ECMP) routing allows routers to distribute traffic across multiple next-hop links based on a hash of fields in the packet header, commonly the five-tuple of source IP, destination IP, source port, destination port, and protocol number. Classic traceroute varies the destination port for each probe, which causes successive probes to potentially follow different ECMP paths. This produces measurement artifacts that Augustin et al. [3] categorized

as loops, cycles, and diamonds — false topological structures that do not exist in the actual forwarding path.

Paris traceroute [3] addresses this by holding the flow identifier constant across all probes to a given destination, ensuring that probes traverse the same ECMP path. This provides a consistent single-path view of the route. Augustin et al. [4] further extended Paris traceroute with the Multipath Detection Algorithm (MDA), which systematically discovers all load-balanced paths at each hop by varying flow identifiers with statistical guarantees.

Our tool uses Paris-mode probing (UDP-Paris and ICMP-Paris) via scamper and cycles through a small set of flow IDs (default: 2) to obtain a minimal ECMP awareness without the full cost of multipath enumeration.

C. Paris Traceroute and ECMP

Equal-Cost Multipath (ECMP) routing allows routers to distribute traffic across multiple next-hop links based on a hash of fields in the packet header, commonly the five-tuple of source IP, destination IP, source port, destination port, and protocol number. Classic traceroute varies the destination port for each probe, which causes successive probes to potentially follow different ECMP paths. This produces measurement artifacts that Augustin et al. [3] categorized as loops, cycles, and diamonds — false topological structures that do not exist in the actual forwarding path.

Paris traceroute [3] addresses this by holding the flow identifier constant across all probes to a given destination, ensuring that probes traverse the same ECMP path. This provides a consistent single-path view of the route. Augustin et al. [4] further extended Paris traceroute with the Multipath Detection Algorithm (MDA), which systematically discovers all load-balanced paths at each hop by varying flow identifiers with statistical guarantees.

Our tool uses Paris-mode probing (UDP-Paris and ICMP-Paris) via scamper and cycles through a small set of flow IDs (default: 2) to obtain a minimal ECMP awareness without the full cost of multipath enumeration.

D. Scamper

Scamper [5] is a scalable and extensible packet prober developed at CAIDA for active Internet measurement. It supports a variety of measurement techniques including traceroute, ping, and alias resolution, with structured JSON output that facilitates automated processing. Scamper natively supports Paris traceroute modes (UDP-Paris, ICMP-Paris, TCP) and is the measurement engine behind CAIDA's Ark platform [2], which continuously maps Internet topology from over 100 globally distributed vantage points.

Our implementation wraps scamper at the single-probe granularity, invoking it once per hop per probe with constrained TTL parameters (-f TTL -m TTL -q 1) and parsing the JSON output to make per-hop decisions before sending the next probe. This single-probe-at-a-time approach is what enables the adaptive budget logic described in Section 3.

E. Related Work on Efficient Probing

Several prior works have addressed the cost of traceroute probing, each targeting a different dimension of redundancy.

Doubletree: Donnet et al. [6] proposed Doubletree, an algorithm that reduces redundancy across multiple vantage points by exploiting the tree-like structure of Internet routes: paths tend to converge near the destination and diverge near the source. Doubletree probes inward from both ends and stops when it encounters a previously discovered node, significantly reducing the total number of probes across a distributed measurement campaign. However, Doubletree targets inter-trace redundancy — the duplication of effort when multiple vantage points probe overlapping path segments. Our work is complementary, focusing on intra-trace efficiency: reducing the number of probes needed for a single trace from a single vantage point.

Diamond-Miner: Vermeulen et al. [7] presented Diamond-Miner, the first system capable of comprehensive discovery of ECMP topology diamonds at Internet scale. Diamond-Miner sends many probes per hop with varying flow identifiers to enumerate all load-balanced next-hops with high statistical confidence. Its goal is completeness of ECMP path discovery. Our work takes the opposite approach, trading completeness for efficiency: we use a minimal flow-ID rotation (2 flow IDs by default) to detect the presence of ECMP without attempting full enumeration, and instead focus on minimizing total probe expenditure.

Adaptive TTL strategies: Other works have explored adaptive techniques such as starting traceroute from a predicted path length and probing backward [8], or using binary search to locate the destination hop. However, these approaches focus on TTL selection strategy rather than per-hop budget adaptation. Existing approaches do not appear to explore per-hop credit redistribution within a single trace, which is the focus of our approach.

Positioning: Table 1 summarizes the relationship between our approach and prior work. Doubletree optimizes across traces from multiple vantage points; Diamond-Miner maximizes ECMP coverage per hop; our Budget Trace optimizes within a single trace by adapting per-hop effort to observed difficulty, using a self-balancing credit pool under a fixed global budget.

Approach	Scope	Goal	Mechanism
Doubletree[6]	Multi-vantage, inter-trace	Reduce redundant probing across VPs	Stop-set sharing
Diamond-Miner[7]	Per-hop, per-destination	Complete ECMP enumeration	Statistical flow-ID probing
Budget-Trace (ours)	Single trace, intra-trace	Minimize probes per trace	Per-hop early + credit pool

Fig: Table 1

III. DESIGN AND ALGORITHM

This section describes the design of Budget Trace, our budget-aware adaptive traceroute algorithm. The core idea is simple: rather than allocating a fixed number of probes to every hop, we assign each hop a base budget, allow hops that

resolve quickly to return their unused probes to a shared credit pool, and permit genuinely difficult hops to draw extra probes from that pool. The entire trace operates under a hard global budget cap, ensuring that the total probe cost never exceeds a predefined limit.

A. Overview

Budget Trace processes hops sequentially, starting from TTL 1 and incrementing toward a configurable maximum TTL (h_{max}). At each hop, the algorithm sends probes one at a time, evaluating the response after each probe to decide whether the hop can be resolved, is unresponsive, or requires further probing. Three outcomes are possible for each hop:

- **Confident:** A single IP address has been observed at least r times (where r is a configurable repeats needed threshold), and the hop is locked to that IP.
- **Dark:** The hop has produced mostly timeouts despite sufficient probing attempts, and is marked as unresponsive ("∅").
- **Undecided:** Neither condition is met, and additional probes are warranted if budget permits.

The trace terminates when one of three conditions is met: (1) a destination reply is received, (2) the global probe budget is exhausted, or (3) the maximum TTL is reached.

B. Per-Hop Budget and Early Termination

Each hop begins with a base per-hop budget B_{base} , which represents the maximum number of probes the hop would receive under normal conditions (analogous to the q parameter in standard traceroute). However, unlike standard traceroute, Budget Trace does not blindly send all B_{base} probes. Instead, it evaluates after each probe and terminates early when a decision can be made.

Confident Rule: A hop is marked as confident when the most frequently observed IP address has been seen at least rr times:

$$confident(h) = \begin{cases} true & \text{if } \maxip(count(h, ip)) \leq r \\ false & \text{otherwise} \end{cases}$$

For example, with $r=2$, if the first two probes at a hop both return the same IP, the hop is immediately locked with only 2 probes used — saving $B_{base}-2$ probes compared to standard traceroute.

Dark Rule: A hop is declared dark (unresponsive) when it has received sufficient probes and nearly all have timed out

$$dark(h) = \begin{cases} true & \text{if } a_h \geq \min(3, C_h) \wedge t_h \geq a_h - 1 \\ true & \text{if } a_h \geq C_h \\ false & \text{otherwise} \end{cases}$$

where a_h is the number of attempts at hop h , t_h is the number of timeouts, and C_h is the dynamic cap for that hop. The first condition provides an aggressive early exit: if at least 3 probes have been sent (or fewer if the cap is below 3) and all but at most one have timed out, the hop is declared dark without exhausting the full budget. The second condition is a safety fallback ensuring that a hop is always finalized once it reaches its cap.

C. The Credit Pool

The credit pool is the central mechanism that distinguishes Budget Trace from a simple early-termination heuristic. It enables cross-hop budget redistribution: probes saved at easy hops are deposited into a shared pool, and difficult hops may withdraw from it.

Deposits: When a hop finalizes using fewer than B_{base} probes, the difference is deposited into the pool:

$$deposit(h) = \max(0, B_{base} - a_h)$$

$$P \leftarrow \min(P + deposit(h), P_{max})$$

where P is the current pool balance and P_{max} is a configurable cap on the pool size (default: 10) to prevent unbounded accumulation.

Withdrawals. When a hop is identified as genuinely uncertain (see Section D), its dynamic cap C_h is increased beyond B_{base} by drawing from the pool:

$$extra(h) = \min(P, W_{max})$$

$$C_h = \min(B_{base} + extra(h), H_{max})$$

where W_{max} is the maximum credits that can be added to a single hop (default: 2) and H_{max} is a hard ceiling on per-hop attempts (default: 6), regardless of pool availability. After the hop finalizes, the actually consumed extra probes are deducted:

$$P \leftarrow \max(0, P - \max(0, a_h - B_{base}))$$

This creates a self-balancing system: a path with many easy hops (e.g., stable routers responding on the first two probes) builds up a pool that funds additional probing at the few difficult hops (e.g., rate-limited or ECMP-heavy routers).

D. Uncertainty Detection and Trouble Zones

Not every hop that has not yet reached confidence should receive extra credits. Budget Trace applies two guards to prevent wasteful spending:

Uncertainty Rule: A hop is considered genuinely uncertain only if it exhibits one of:

- **Heavy silence:** 2 or more timeouts observed so far, suggesting an unresponsive or rate-limited router.
- **Heavy ECMP:** 3 or more distinct IP addresses observed, suggesting significant load balancing.

Depth Gate: Even if a hop is uncertain, extra credits are only granted if:

1. The hop is deep in the path ($TTL > 6$), avoiding spending extra probes on near-source hops where instability is typically transient.

2. The previous hop was itself dark ("∅") or noisy ($t_{h-1} \geq 2$), indicating a trouble zone — a contiguous region of the path where routers are unresponsive or inconsistent.

This two-part guard ensures that the credit pool is spent where it is most likely to be useful: in the interior of the network where dark or noisy regions tend to cluster.

E. ECMP - Aware Flow Cycling

To provide minimal awareness of load-balanced paths without the cost of full ECMP enumeration (as in Diamond-Miner [7]), Budget Trace cycles through a small set of flow

identifiers. For each probe at a hop, the flow ID is selected by round-robin:

$$f_h = \text{flow_ids}[a_h \bmod |\text{flow_ids}|]$$

With the default configuration of two flow IDs (0 and 1), alternating probes at the same hop may traverse different ECMP paths, allowing the algorithm to detect the presence of load balancing (multiple distinct IPs in the hop's count table) without dedicating probes specifically to ECMP discovery.

F. Algorithm/Pseudocode

Algorithm : Budget Trace

Input: dest — target IP address
 B_base — base per-hop budget (default: 6)
 r — repeats needed for confidence (default: 2)
 B_total — global probe budget (default: 120)
 h_max — maximum TTL (default: 32)
 flow_ids — set of flow IDs (default: {0, 1})
 W_max — max rollover credits per hop (default: 2)

2) H_max — hard per-hop ceiling (default: 6)
 P_max — max pool size (default: 10)

Output: path — map of TTL → IP (or $\emptyset$)
 probes_used — total probes sent
 stop_reason — why the trace ended

```

1: pool ← 0
2: probes_used ← 0
3: path ← {}
4:
5: for ttl ← 1 to h_max do
6:   if probes_used ≥ B_total then
7:     stop_reason ← "budget_exhausted"
8:     break
9:
10:  counts ← {}      ▷ IP → count for this hop
11:  timeouts ← 0
12:  attempts ← 0
13:
14:  // Compute dynamic cap
15:  C ← B_base
16:  if uncertain(ttl) AND ttl > 6 AND prev_hop_noisy
then
17:    extra ← min(pool, W_max)
18:    C ← min(B_base + extra, H_max)
19:
20:  // Probe loop for this hop
21:  while attempts < C AND probes_used < B_total do
22:    flow ← flow_ids[attempts mod |flow_ids|]
23:    event ← probe_once(dest, ttl, flow)
24:    probes_used ← probes_used + 1
25:    attempts ← attempts + 1
26:
27:    if event.status ∈ {ttl_exceeded, dest_reached} then
28:      counts[event.hop_ip] ← counts[event.hop_ip] + 1

```

```

29:    if event.status = dest_reached then
30:      path[ttl] ← event.hop_ip
31:      stop_reason ← "dest_reached"
32:      return path, probes_used, stop_reason
33:    else
34:      timeouts ← timeouts + 1
35:
36:    // Check early termination
37:    if confident(counts, r) then
38:      path[ttl] ← argmax_ip(counts)
39:      break
40:    if dark(timeouts, attempts, C) then
41:      path[ttl] ←  $\emptyset$ 
42:      break
43:
44:  // Update credit pool
45:  if attempts < B_base then
46:    deposit ← B_base - attempts
47:    pool ← min(pool + deposit, P_max)
48:  else if attempts > B_base then
49:    withdrawn ← attempts - B_base
50:    pool ← max(0, pool - withdrawn)
51:
52: stop_reason ← "max_ttl"
53: return path, probes_used, stop_reason

```

G. Parameter Summary

Parameter	Symbol	Default	Role
Base per-hop budget	B_base	6	Maximum probes per hop without rollover
Repeats needed	r	2	Same-IP replies to lock a hop
Global budget	B_total	75	Hard cap on total probes per trace
Maximum TTL	h_max	32	Deepest hop to probe
Flow IDs		{0, 1}	Flow identifiers for ECMP cycling
Rollover cap per hop	W_max	2	Max extra credits from pool per hop
Hard per-hop ceiling	H_max	6	Absolute max attempts even with rollover
Pool maximum	P_max	10	Cap on accumulated pool credits
Probe pacing		30ms	Minimum inter-probe delay

Fig: Table 2

H. Correctness and Budget Guarantee

The algorithm maintains the following invariant: at all times, $\text{probes_used} \leq B_{\text{total}}$. This is enforced by the guard at lines 6 and 21. The credit pool is a redistribution mechanism only — it never creates new probes. Credits deposited at line 46 represent probes that were allocated but not sent; credits withdrawn at line 17–18 allow sending additional probes only up to H_{max} per hop.

Therefore:

$$\sum_{h=1}^{hmax} ah = probes_used \leq Btotal$$

The pool balance P satisfies $0 \leq P \leq Pmax$ at all times. In the worst case where no hop terminates early, the algorithm degenerates to standard traceroute with B_{base} probes per hop and an empty pool, ensuring that it never performs worse than a fixed-budget baseline (aside from the overhead of single-probe invocation).

IV. IMPLEMENTATION

Budget Trace is implemented in Python and is structured as a modular, testable system with three main components: the brain (decision logic), the prober (network interface), and the CLI (user entry points).

A. Architecture

The system follows a clean separation between decision logic and probe execution. The BudgetController class implements the main loop described in Section III. It receives a Prober object and a Settings configuration, and is agnostic to how probes are actually sent. This separation allows the same controller to operate with a real network prober or a deterministic fake prober for testing.

The Prober interface defines a single method, `probe_once(dest, ttl, flow_id)`, which sends exactly one probe and returns a structured ProbeEvent dictionary containing the reply status, responding IP, RTT, and raw data. Two implementations are provided:

- ScamperProber: wraps the scamper binary [5], invoking it once per probe with constrained TTL parameters and parsing the JSON output. It handles sudo privilege escalation, pacing with jitter, and classification of ICMP response types (Time Exceeded, Destination Unreachable, Echo Reply) across UDP-Paris and ICMP-Paris modes.
- FakeProber: accepts a scripted sequence of probe responses, enabling deterministic unit testing of the controller logic without network access.

B. Probe Execution

Each probe is executed as a separate scamper invocation with the first-hop and max-hop both set to the target TTL (`-f TTL -m TTL`) and a single attempt (`-q 1`). While this per-probe invocation has higher overhead than a bulk trace, it is essential for the adaptive logic: the controller must observe each response before deciding whether to continue probing the current hop, advance to the next, or terminate the trace.

Inter-probe pacing is enforced with a configurable base delay (default: 30 ms) plus random jitter (± 10 ms) to avoid triggering ICMP rate limiting on intermediate routers.

C. Configuration

All algorithm parameters (Table 2 in Section III) are encapsulated in a Settings dataclass with sensible defaults. Parameters can be overridden via command-line arguments, enabling easy experimentation with different budget configurations.

D. Testing

The FakeProber abstraction allows the controller to be tested in isolation. Unit tests script specific network scenarios — stable paths, dark hops, ECMP — and verify that the controller produces the expected path, respects the budget, and correctly manages the credit pool, all without requiring network access or elevated privileges.

V. EVALUATION

This section evaluates Budget Trace against standard traceroute across 85 real-world Internet targets. We assess two primary questions: (1) how much probe reduction does Budget Trace achieve, and (2) how much path accuracy is preserved?

A. Experimental Setup

Measurement platform: All experiments were conducted from a single vantage point running Linux, using scamper [5] as the underlying probing engine. Probes were sent using ICMP-Paris mode to maintain flow-consistent paths across ECMP routers.

Targets: We selected 85 IPv4 destinations spanning a diverse range of Internet infrastructure:

Category	Count	Examples
Public DNS resolvers	16	Google (8.8.8.8), Cloudflare (1.1.1.1), Quad9, OpenDNS, Level3, Comodo, Neustar, CleanBrowsing, AdGuard, Yandex
Web/Social Media	15	Google, Meta/Facebook, X/Twitter, Wikipedia, Yahoo, Reddit, LinkedIn
Cloud Providers	18	AWS, GCP, Microsoft Azure, AWS CloudFront, AWS Global Accelerator
CDN infrastructure	14	Cloudflare, Akamai, Fastly, Edgecast/Verizon
Other Services	22	GitHub, Dropbox, Apple, Stack Overflow, Imgur

This selection was designed to cover paths of varying lengths, stability, and ECMP complexity.

Baseline: For each target, we ran a standard scamper trace with $q=3$ probes per hop, `gaplimit` of 10, and `gap action` of 2 — representative of conventional traceroute practice. The baseline trace determines the maximum TTL probed, and the same maximum TTL is used for the corresponding Budget Trace run, ensuring a fair comparison over the same path depth.

Budget Trace configuration: We used the following parameters: $B_{\text{base}}=3$, $r=2$, $B_{\text{total}}=65$ and flow IDs = $\{0, 1\}$. The base per-hop budget was set to 3 to match the baseline's probes per hop, making the comparison direct: any reduction comes purely from the adaptive early termination and credit pooling logic.

B. Metrics

Path accuracy. For each target, we compare the Budget Trace path against the baseline path hop by hop. Only hops where the baseline returned a non-empty IP (i.e., not dark/"∅") are considered. Accuracy is the fraction of these comparable hops where Budget Trace produced the same IP:

$$\text{Accuracy} = \frac{|\{h: \text{budget}(t,h) = \text{baseline}(t,h)\}|}{|\{h: \text{baseline}(t,h) \neq \emptyset\}|} \times 100\%$$

Probe count. The total number of probes sent by each method.

Probe reduction. The percentage decrease in probes relative to the baseline:

$$\text{Reduction} = \frac{\sum_t \text{Preg}(t) - \sum_t \text{Pbud}(t)}{\sum_t \text{Preg}(t)} \times 100\%$$

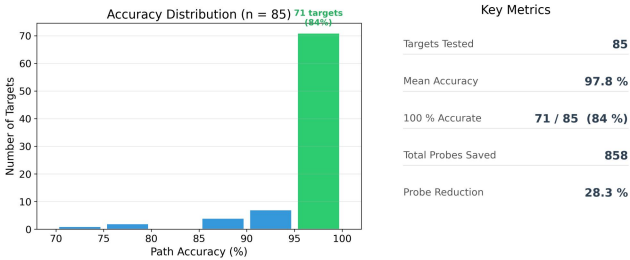
C. Overall Results

Metric	Value
Targets Tested	85
Mean Path Accuracy	97.83%
Targets with 100% accuracy	71/85 (83.5%)
Targets with $\geq 90\%$ accuracy	78/85 (91.8%)
Total baseline probes	3027
Total Budget Trace Probes	2169
Aggregate Probe Reduction	28.34%

Budget Trace reduces total probe expenditure by over 28% while maintaining an average path accuracy of nearly 98%. Over four out of five targets are traced with perfect accuracy, and over 91% of targets achieve 90% accuracy or higher. These key metrics are also presented visually in the right panel of Figure 2.

D. Accuracy Analysis

The bar chart(left panel) shows the distribution of per-target accuracy. The vast majority of targets cluster at or above 95%, with a small tail of reduced accuracy concentrated on specific network types.



The 14 targets with accuracy below 100% fall into two categories:

Microsoft / Azure (8 targets, accuracy 73–92%): These paths exhibited the lowest accuracy, with individual targets

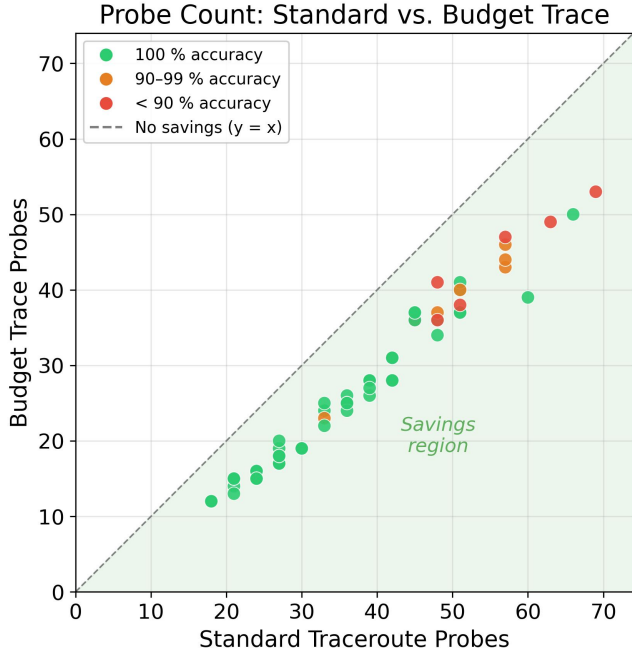
ranging from 73.3% to 91.7%. Microsoft's network is known for heavy ECMP load balancing and per-flow path diversity, meaning that even with Paris-mode probing, different probing runs may observe different router interfaces at the same TTL. When the baseline and Budget Trace observe different ECMP members at a hop, this registers as a mismatch even though both IPs are valid.

Target	Accuracy	Baseline Probes	Budget Probes
20.112.250.133	73.3%	63	49
20.190.151.132	75.0%	57	47
162.125.66.1 (Dropbox)	77.8%	48	41
40.90.4.1	85.7%	48	36
162.125.1.1 (Dropbox)	86.7%	51	38
20.53.203.50	88.2%	69	53
140.82.112.4 (GitHub)	88.9%	45	36
204.79.197.200	90.9%	51	40
149.112.112.112 (Quad9)	90.9%	42	31
4.2.2.2 (Level3)	90.0%	33	23
13.107.42.14	90.9%	48	37
20.190.151.70	91.7%	57	46
23.192.228.80 (Akamai)	92.9%	57	43
23.4.43.67 (Akamai)	92.3%	57	44

Importantly, these accuracy reductions are largely attributable to ECMP-induced measurement variability rather than algorithmic error: different probing runs naturally observe different load-balanced interfaces. Budget Trace's reduced probing means fewer samples per hop, which slightly increases susceptibility to this effect.

E. Probe Saving Analysis

Figure 1 presents a scatter plot of baseline probes versus Budget Trace probes for all 85 targets. Points below the dashed $y=x$ line indicate probe savings; the farther below, the greater the reduction. Nearly all targets fall within the savings region, confirming consistent reductions across the dataset. Points are colored by accuracy: green for 100%, orange for 90–99%, and red for below 90%.



Probe savings vary with path length and path stability.

Short, stable paths show the highest relative savings. Targets reachable in 6–8 hops (e.g., AWS, GCP, Apple, Cloudflare) typically use only 12–17 Budget Trace probes versus 18–27 baseline probes, as most hops resolve in 2 probes and donate their surplus to the pool

Target	Path Hops	Baseline	Budget	Reduction
205.251.242.103 (AWS)	6	18	12	33.3%
35.190.247.0 (GCP)	7	21	13	38.1%
17.253.144.26 (Apple)	6	18	12	33.3%
93.184.216.34 (example.com)	7	21	15	28.6%

Long, complex paths show lower but still meaningful savings. Targets with 15+ hops (e.g., Yandex, Meta, Twitter) use 37–50 Budget Trace probes versus 51–66 baseline probes. The longer path provides more opportunities for early termination, but also more hops that may require full probing.

Target	Path Hops	Baseline	Budget	Reduction
205.251.242.103 (AWS)	17	51	37	27.5%
35.190.247.0 (GCP)	22	66	50	24.2%
17.253.144.26 (Apple)	17	51	40	21.6%

Paths with dark hops benefit significantly from early dark detection. When routers do not respond, the dark rule (Section 3.2) finalizes the hop in as few as 3 probes rather

than the full B_{base} , and the saved probes are deposited into the credit pool.

F. Summary of Findings

1. Budget Trace achieves a 28.34% reduction in total probes across 85 diverse Internet targets.
2. 97.83% mean path accuracy is maintained, with 83.5% of targets traced with perfect accuracy.
3. Accuracy reductions are concentrated on networks with heavy ECMP load balancing (Microsoft/Azure, Dropbox), where measurement variability is inherent regardless of methodology.
4. The credit pool mechanism effectively redistributes budget from easy hops to difficult ones, enabling the algorithm to maintain accuracy without exceeding the global budget.
5. Savings are consistent across path lengths, with short stable paths showing up to 38% reduction and long complex paths still achieving over 20%.
6. As shown in Figure 1, no target required more probes than the baseline — Budget Trace is never worse than standard traceroute in terms of probe cost.

VI. DISCUSSION

A. Why Accuracy Drops on Some Targets

The targets with the lowest accuracy — primarily Microsoft/Azure and Dropbox — share a common trait: heavy ECMP load balancing deep in the path. In these networks, routers distribute traffic across multiple next-hops based on flow-header hashes, meaning that even Paris-mode probing does not guarantee the same interface is seen across two separate measurement runs.

This is not a flaw of Budget Trace specifically. If the baseline and Budget Trace are run sequentially (as in our evaluation), the underlying ECMP mapping may shift between runs due to routing changes, flow-table rehashing, or router restarts. With fewer probes per hop, Budget Trace has fewer samples to "vote" on the majority IP, making it marginally more sensitive to this inter-run variability. A fairer evaluation would run both methods simultaneously with identical flow tuples, but this is not possible when comparing against a standard bulk scamper trace.

We note that all "mismatched" hops on these targets still returned valid router IPs — they were simply different ECMP members, not incorrect results. In topology mapping applications where any valid next-hop interface is acceptable, the effective accuracy would be higher than reported.

B. Parameter Sensitivity

The algorithm's behavior is governed by several interacting parameters. We discuss the key tradeoffs:

Repeats needed (r). Setting $r=2$ means a hop is locked after two identical replies. This is aggressive — it maximizes probe savings but offers weaker confidence than $r=3$. In our experiments, this was sufficient for 83.5% of targets to achieve perfect accuracy. Increasing r to 3 would improve confidence on ECMP-heavy paths at the cost of reduced savings, since every hop would require at least 3 probes.

Base per-hop budget (B_{base}). Our evaluation used $B_{\text{base}}=3$ to match the baseline's $q=3$. The algorithm supports higher values (up to 6 in our default configuration), which would provide more headroom for ECMP detection but reduce the savings margin. The choice of B_{base} should reflect the measurement goal: topology surveys that prioritize coverage can use $B_{\text{base}}=2$, while studies requiring high per-hop confidence can use $B_{\text{base}}=4$ or higher.

Depth gate ($\text{TTL} > 6$). The threshold of 6 for the depth gate was chosen heuristically based on the observation that near-source hops (the first few routers in a campus or ISP edge network) tend to be stable and well-provisioned, rarely benefiting from extra probes. Lowering this threshold would make the credit pool more generous but risks spending credits on hops that do not need them. In future work, this threshold could be made adaptive based on observed path characteristics.

Rollover cap ($W_{\text{max}}=2$) and hard ceiling ($H_{\text{max}}=6$). These parameters prevent a single difficult hop from consuming the entire pool. The conservative defaults ensure that the pool is spread across multiple trouble hops rather than exhausted on one. In practice, with $W_{\text{max}}=2$ and $H_{\text{max}}=6$, the maximum extra probes any hop can receive is 2, which is sufficient to break ties in moderately noisy conditions but not enough for full ECMP enumeration.

C. Scalability Considerations

Budget Trace sends probes one at a time and makes a decision after each response. This sequential approach is inherently slower than a bulk trace that sends all probes for all TTLs in parallel (as scamper can do in its default mode). For a single trace, this overhead is modest — the 30 ms inter-probe pacing dominates, and a 20-probe trace completes in roughly 0.6 seconds.

At scale, however, the per-probe invocation of scamper introduces process startup overhead. Each probe requires spawning a new scamper process, parsing its JSON output, and making a decision. For large-scale measurement campaigns (thousands of targets), this could be mitigated by:

- Integrating the decision logic directly into scamper's probing loop rather than wrapping it externally.
- Using scamper's built-in scripting capabilities or the recently introduced Python bindings [9] to avoid per-probe process spawning.
- Parallelizing across targets (each target is independent) while keeping per-target probing sequential.

D. Limitations

Single vantage point: All experiments were conducted from a single location. Path characteristics (length, ECMP prevalence, dark hops) vary significantly by vantage point and geographic region. The reported savings and accuracy may differ from other vantage points.

No full ECMP discovery: Budget Trace uses only 2 flow IDs and does not attempt to enumerate all load-balanced paths. For applications requiring complete ECMP topology (e.g., topology diamond detection), tools like Diamond-

Miner [7] remain necessary. Budget Trace is designed for the more common case of efficient single-path discovery.

Temporal variability: Our evaluation compares two sequential runs. Routing changes between runs may cause mismatches that are neither the fault of Budget Trace nor the baseline. Repeated measurements over time would provide a more robust accuracy estimate.

Parameter tuning: The current parameter defaults were selected based on engineering judgment and informal experimentation. A systematic sensitivity analysis across a larger target set and multiple vantage points would help identify optimal configurations for different measurement scenarios.

VII. CONCLUSION AND FUTURE WORK

A. Conclusion

This paper presented Budget Trace, a budget-aware adaptive traceroute algorithm that reduces probe expenditure by dynamically allocating per-hop effort based on observed difficulty. The key insight is that most hops along an Internet path resolve quickly — often in just two probes — while a small number of hops are genuinely difficult due to ECMP load balancing or unresponsive routers. By allowing easy hops to return their unused probes to a shared credit pool and permitting difficult hops to draw from it, Budget Trace achieves significant savings without sacrificing path accuracy.

We evaluated Budget Trace against standard scamper traceroute across 85 diverse Internet targets spanning DNS resolvers, CDN infrastructure, cloud providers, social media platforms, and other services. The results demonstrate:

- A 28.34% reduction in total probes (2,169 vs. 3,027).
- A 97.83% mean path accuracy, with 83.5% of targets traced with perfect accuracy.
- Consistent savings across path lengths, from 38% on short stable paths to over 20% on long complex paths.
- No target required more probes than the baseline — Budget Trace is never worse than standard traceroute in terms of probe cost.

The algorithm is simple to implement, protocol-agnostic, and requires no coordination with other vantage points, making it suitable as a drop-in improvement for existing measurement workflows.

B. Future Work

Several directions remain for extending this work:

Multi-vantage evaluation: Our experiments used a single vantage point. Evaluating Budget Trace from geographically diverse locations — using platforms such as CAIDA Ark [2] or RIPE Atlas — would characterize how savings and accuracy vary with network topology and regional routing practices.

Systematic parameter tuning: The current defaults ($r=2$, $B_{\text{base}}=3$ –66, TTL depth gate = 6) were selected based on engineering judgment. A systematic sensitivity analysis across a large target set would help identify optimal parameter configurations for different measurement goals, such as maximizing savings versus maximizing accuracy.

Adaptive depth gate: The fixed TTL > 6 threshold for credit pool withdrawals could be replaced with an adaptive criterion — for example, triggering based on a rolling window of dark or noisy hops rather than a fixed depth. This would allow the algorithm to respond to trouble zones regardless of where they appear in the path.

Native scamper integration: The current implementation spawns a separate scamper process per probe. Integrating the decision logic directly into scamper — either through its scripting interface or the recently introduced Python bindings [9] — would eliminate per-probe process overhead and make Budget Trace practical for large-scale campaigns involving thousands of targets.

Parallelization: Since each target trace is independent, a batch mode could run multiple Budget Trace instances in parallel across a target list while keeping per-target probing sequential. Combined with native scamper integration, this would enable deployment at measurement infrastructure scale.

Broader applicability: The credit pool mechanism is not specific to traceroute. Any sequential measurement task where individual steps have variable difficulty could benefit from adaptive budget redistribution. Potential applications include DNS resolution chains, TLS certificate probing, and BGP path exploration, where some targets resolve quickly and others require repeated queries.

ECMP-aware extensions: The current 2-flow-ID rotation provides minimal ECMP detection. A natural extension would be an adaptive flow strategy that increases the number of flow IDs when load balancing is detected at a hop, providing richer topology information at the cost of additional probes — effectively bridging the gap between

Budget Trace's efficiency and Diamond-Miner's [7] completeness.

VIII. ACKNOWLEDGEMENT

The author would like to thank Dr. Khaled Sabehein for their guidance and supervision throughout this project.

We would also like to thank the Mathematics, Computer, and Information Sciences Department at Mississippi Valley State University for funding this research.

REFERENCES

- [1] V. Jacobson, traceroute, 1989. Software tool. Available: <ftp://ftp.ee.lbl.gov/traceroute.tar.gz>
- [2] CAIDA, "The CAIDA UCSD Archipelago (Ark) Measurement Infrastructure." Available: <https://www.caida.org/projects/ark/>
- [3] B. Augustin, X. Cuvelier, B. Orgogozo, F. Viger, T. Friedman, M. Latapy, and R. Teixeira, "Avoiding traceroute anomalies with Paris traceroute," in Proc. ACM IMC, 2006.
- [4] B. Augustin, T. Friedman, and R. Teixeira, "Measuring load-balanced paths in the Internet," in Proc. ACM IMC, 2007.
- [5] M. Luckie, "Scamper: a scalable and extensible packet prober for active measurement of the internet," in Proc. ACM IMC, 2010.
- [6] B. Donnet, P. Raoult, T. Friedman, and M. Crovella, "Efficient algorithms for large-scale topology discovery," in Proc. ACM SIGMETRICS, 2005.
- [7] K. Vermeulen, J.P. Rohrer, R. Beverly, O. Fourmaux, and T. Friedman, "Diamond-Miner: Comprehensive discovery of the internet's topology diamonds," in Proc. USENIX NSDI, 2020.
- [8] T. Moors, "Streamlining traceroute by estimating path lengths," in Proc. IEEE IPOM, 2004.
- [9] M. Luckie, S. Hariprasad, R. Sommesse, B. Jones, and E. Aben, "An Integrated Active Measurement Programming Environment," in Proc. Passive and Active Network Measurement (PAM), 2025.